Tramos, matrices y recursión: soluciones

Programación: UVa 507, UVa 108 y UVa 839

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz

3 Recursión sobre un árbol

Contenido

1 El mejor tramo de un arreglo
- Solución: UVa 507

2 Rectángulos en una matriz

3 Recursión sobre un árbol

Primera idea: todos los tramos

Python: `jill_acumulados.py`, con la suma de cada tramo en $O(1)$

```python
def mejor_tramo(v):
    acum = [0]
    for x in v:
        acum.append(acum[-1] + x)
    mejor = 0
    mi = 0
    mj = -1
    for i in range(len(v)):
        for j in range(i, len(v)):
            suma = acum[j + 1] - acum[i]
            if suma > mejor or (suma == mejor and j - i > mj - mi):
                mejor = suma
                mi = i
                mj = j
    return mejor, mi, mj
```

El desempate sale del orden: i crece afuera y la comparación de largo es estricta, así que entre dos tramos iguales en suma y en largo queda el de menor i.

Primera idea: todos los tramos

¿Cuánto cuesta?

Los tramos son $(s-1)s/2$, unos $2 \cdot 10^8$ con $s = 20\,000$. Sin acumulados, cada suma cuesta además hasta s pasos y el total pasa a $O(s^3)$.

Medido (una ruta; Python 3.14, Ryzen 7 7435HS)

s	todos los tramos	un recorrido
5 000	1,29 s	0,02 s
10 000	5,11 s	0,02 s
20 000	20,77 s	0,02 s

Contra los límites

Veinte segundos por una sola ruta de tamaño máximo, contra 3 segundos de límite para el archivo entero, que trae varias.

¿Sirve lo que se lleva sumado?

Pregunta

Se recorre la ruta llevando la suma del tramo que empezó en `inicio` y termina en la calle actual k. Si esa suma se vuelve negativa, ¿le conviene a algún tramo que siga después de k arrastrarla?

Con $v = [4, -5, 4, \ldots]$

Después de $4 - 5$ la suma es -1. Un tramo que empiece en la calle 2 vale lo mismo que con ese -1 delante, más uno.

¿Sirve lo que se lleva sumado?

Respuesta

No. Un prefijo negativo solo resta: la suma vuelve a 0 y el tramo empieza en $k + 1$. Es la idea de Kadane.

El recorrido, en palabras: $\Theta(s)$ por ruta

1. Sumar $v[k]$ a la suma actual.
2. Si la suma supera a la mejor, o la iguala con un tramo más largo, guardar el tramo $[inicio, k]$.
3. Si la suma quedó negativa, volver a 0 y poner $inicio = k + 1$.

¿Negativa o no positiva?

Se reinicia con $suma < 0$, no con $suma \leq 0$. Un prefijo que suma cero no resta, pero alarga el tramo, y el enunciado premia el tramo largo.

Traza con la ruta 2

k	$v[k]$	`suma`	`inicio`	`mejor`	tramo $[mi, mj]$
0	4	4	0	4	$[0, 0]$
1	-5	$-1 \to 0$	$0 \to 2$	4	$[0, 0]$
2	4	4	2	4	?
3	-3	?	?	?	?
4	4	?	?	?	?
5	4	?	?	?	?
6	-4	?	?	?	?
7	4	?	?	?	?
8	-5	?	?	?	?

Pregunta

En $k = 2$ la suma vuelve a 4, igual que la mejor. ¿Cambia el tramo guardado?

Traza con la ruta 2

k	$v[k]$	`suma`	`inicio`	`mejor`	tramo $[mi, mj]$
0	4	4	0	4	$[0, 0]$
1	-5	$-1 \to 0$	$0 \to 2$	4	$[0, 0]$
2	4	4	2	4	$[0, 0]$, mismo largo
3	-3	1	2	4	$[0, 0]$
4	4	5	2	5	$[2, 4]$
5	4	9	2	9	$[2, 5]$
6	-4	5	2	9	$[2, 5]$
7	4	9	2	9	$[2, 7]$, más largo
8	-5	4	2	9	$[2, 7]$

Respuesta

En $k = 2$ no: el tramo $[2, 2]$ tiene el mismo largo que $[0, 0]$ y empieza después.
En $k = 7$ sí: suma 9 otra vez, con más calles. Calles $[2, 7]$ son las paradas 3 a 9.

¿Qué mantiene el ciclo?

Pregunta

En la vuelta k, justo después de sumar $v[k]$ y comparar, ¿qué es `suma` y qué es `mejor`? Mire la traza: la columna de `suma` y la de `mejor` responden algo distinto.

¿Qué mantiene el ciclo?

Respuesta

`suma` es la suma del mejor tramo que termina en la calle k, y `mejor` la del mejor tramo visto hasta k, con sus extremos en $[mi, mj]$. La vuelta siguiente solo necesita esas dos cosas, y al salir del ciclo `mejor` ya es la respuesta de la ruta.

El mismo par que en UVa 11078

Allá eran `mayor` y `mejor`; aquí `suma` y `mejor`. Una variable resume lo que termina en la casilla actual y la otra todo lo visto.

Python: jill-rides-again.py

El recorrido

```
def mejor_tramo(v):
    # v[k] es la calle entre las paradas k + 1 y k + 2
    suma = 0
    inicio = 0
    mejor = 0
    mi = 0
    mj = -1
    for k in range(len(v)):
        suma = suma + v[k]
        if suma > mejor or (suma == mejor and k - inicio > mj - mi):
            mejor = suma
            mi = inicio
            mj = k
        if suma < 0:
            suma = 0
            inicio = k + 1
    return mejor, mi, mj
```

Python: jill-rides-again.py

La lectura y la salida

```
def main():
    datos = sys.stdin.read().split()
    b = int(datos[0])
    pos = 1
    for r in range(1, b + 1):
        s = int(datos[pos])
        v = []
        for k in range(s - 1):
            v.append(int(datos[pos + 1 + k]))
        pos = pos + s
        mejor, mi, mj = mejor_tramo(v)
        if mejor > 0:
            print("The nicest part of route %d is between stops %d and %d"
                  % (r, mi + 1, mj + 2))
        else:
            print("Route %d has no nice parts" % r)


main()
```

C++: jill-rides-again.cpp

El recorrido

```cpp
long long mejor_tramo(const vector<int>& v, int& mi, int& mj) {
    long long suma = 0;
    int inicio = 0;
    long long mejor = 0;
    mi = 0;
    mj = -1;
    for (int k = 0; k < (int) v.size(); k++) {
        suma = suma + v[k];
        if (suma > mejor || (suma == mejor && k - inicio > mj - mi)) {
            mejor = suma;
            mi = inicio;
            mj = k;
        }
        if (suma < 0) {
            suma = 0;
            inicio = k + 1;
        }
    }
    return mejor;
}
```

C++: jill-rides-again.cpp

```cpp
int main() {
    int b;
    scanf("%d", &b);
    for (int r = 1; r <= b; r++) {
        int s;
        scanf("%d", &s);
        vector<int> v(s - 1);
        for (int k = 0; k < s - 1; k++) {
            scanf("%d", &v[k]);
        }
        int mi, mj;
        long long mejor = mejor_tramo(v, mi, mj);
        if (mejor > 0) {
            printf("The nicest part of route %d is between stops %d and %d\n",
                   r, mi + 1, mj + 2);
        } else {
            printf("Route %d has no nice parts\n", r);
        }
    }
    return 0;
}
```

Trampas

Reiniciar con *suma* ≤ 0

Con $v = [1, -1, 5]$, los tramos $[0, 2]$ y $[2, 2]$ suman 5, y gana el largo: paradas 1 a 4. Si la suma se reinicia al llegar a cero, en $k = 1$ el tramo arranca de nuevo en la calle 2 y la respuesta sale de la parada 3 a la 4.

Comparar solo la suma

Con `suma > mejor` y nada más, la ruta 2 de la muestra da paradas 3 a 7.

La salida exacta

Paradas desde 1: la parada final es $mj + 2$. Cuando la mejor suma no es positiva va la otra frase, y la ruta se numera aunque no tenga tramo.

Variante para pensar

Ruta circular

Después de la última parada el bus vuelve a la primera, así que un tramo puede dar la vuelta: terminar en la calle 0 después de pasar por la última. ¿Cómo se halla la mejor suma con un recorrido?

Pista

Un tramo que da la vuelta deja por fuera un tramo del medio. Lo que queda es la suma total menos ese tramo: hay que quitar el de suma mínima, que también se halla con un recorrido.

¿Cuánto cuesta la solución?

De qué depende

De s, el número de paradas de una ruta, con $s \leq 20\,000$. El archivo no dice cuántas rutas trae, así que el costo se da por ruta.

Cuántas veces corre cada ciclo

La lectura y el ciclo de `mejor_tramo` corren $s - 1$ veces cada uno, con un trabajo fijo por vuelta: sumar, comparar y, a veces, guardar el tramo o reiniciar la suma.

Total, contra el límite

$\Theta(s)$ por ruta y $\Theta(s)$ de memoria para las calles. Con $s = 20\,000$ son unas $2 \cdot 10^4$ vueltas, frente a los $2 \cdot 10^8$ tramos de la primera idea.

El juez da 3 segundos para el archivo entero, y la primera idea gasta 20 en una sola ruta de tamaño máximo.

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz
- Solución: UVa 108

3 Recursión sobre un árbol

Primera idea: todos los rectángulos

¿Cuántos hay?

Un rectángulo es un par de filas $f_1 \leq f_2$ y un par de columnas $c_1 \leq c_2$: $(N(N+1)/2)^2$. Con $N = 100$, $5050^2 \approx 2{,}6 \cdot 10^7$.

Sumando cada uno celda por celda

Cuatro ciclos para elegirlo y dos más para sumarlo: $O(N^6)$, del orden de 10^{10} sumas con $N = 100$. No pasa.

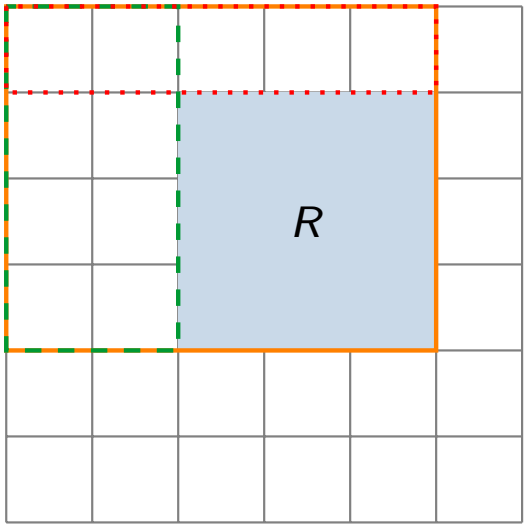
Pregunta

La suma de un tramo sale en $O(1)$ con acumulados. ¿Hay acumulados de un rectángulo?

Acumulados en dos dimensiones

Respuesta

Sí. $P[i][j]$ es la suma del rectángulo que va de la esquina $(0, 0)$ hasta la celda $(i-1, j-1)$, con P en ceros en la fila 0 y la columna 0. Un rectángulo cualquiera sale de cuatro esquinas:



$$R = P[f_2{+}1][c_2{+}1] - P[f_1][c_2{+}1] - P[f_2{+}1][c_1] + P[f_1][c_1]$$

La esquina de arriba a la izquierda se resta dos veces, por eso vuelve a sumarse.

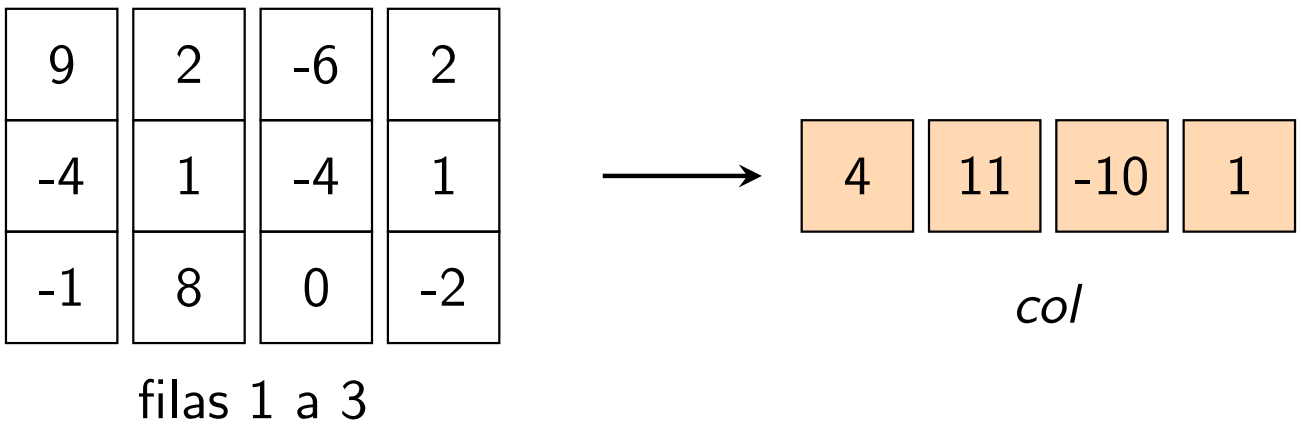
Costo

P se llena en $\Theta(N^2)$; cada rectángulo cuesta $O(1)$; todos, $O(N^4)$: unas $2{,}6 \cdot 10^7$ consultas, mil veces menos que antes.

¿Ya resolvimos esto?

Pregunta

Fije las filas `arriba` y `abajo`. Cada rectángulo entre ellas es un tramo de columnas. Si cada columna se cambia por su suma entre esas filas, ¿qué problema queda?



¿Ya resolvimos esto?

Respuesta

El del mejor tramo: el de suma máxima de *col*. Con $[4, 11, -10, 1]$ es $4 + 11 = 15$, el mismo rectángulo de la muestra.

El algoritmo

1. Para cada `arriba`, poner *col* en ceros.
2. Para cada `abajo` desde `arriba`: sumar la fila `abajo` a *col*, como en los acumulados por columna, y pasar *col* por el recorrido de Kadane.
3. Quedarse con el mayor resultado.

Traza con la muestra, *arriba* $= 1$

Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

`abajo`	*col*	mejor tramo de *col*
1	$[9, 2, -6, 2]$	$9 + 2 = 11$
2	$[5, 3, -10, 3]$	?
3	$[4, 11, -10, 1]$	?

Traza con la muestra, *arriba* $= 1$

Kadane sobre *col*, sin desempates

Aquí solo importa la suma, y puede ser negativa: `mejor` arranca en *col*[0], no en 0.

`abajo`	*col*	mejor tramo de *col*
1	$[9, 2, -6, 2]$	$9 + 2 = 11$
2	$[5, 3, -10, 3]$	$5 + 3 = 8$
3	$[4, 11, -10, 1]$	$4 + 11 = 15$

Resultado

El mejor de los diez pares de filas es 15, con `arriba` $= 1$ y `abajo` $= 3$.

Python: maximum-sum.py

El tramo de suma máxima

```
def mejor_tramo(v):
    mejor = v[0]
    suma = 0
    for x in v:
        suma = suma + x
        if suma > mejor:
            mejor = suma
        if suma < 0:
            suma = 0
    return mejor
```

Python: `maximum-sum.py`

Fijar dos filas

```
def mayor_rectangulo(m):
    n = len(m)
    mejor = m[0][0]
    for arriba in range(n):
        col = [0] * n
        for abajo in range(arriba, n):
            for c in range(n):
                col[c] = col[c] + m[abajo][c]
            s = mejor_tramo(col)
            if s > mejor:
                mejor = s
    return mejor
```

Python: `maximum-sum.py`

La lectura por palabras

```
def main():
    datos = sys.stdin.read().split()
    n = int(datos[0])
    m = []
    for i in range(n):
        fila = []
        for j in range(n):
            fila.append(int(datos[1 + i * n + j]))
        m.append(fila)
    print(mayor_rectangulo(m))


main()
```

C++: maximum-sum.cpp

El tramo de suma máxima

```
int mejor_tramo(const vector<int>& v) {
    int mejor = v[0];
    int suma = 0;
    for (int c = 0; c < (int) v.size(); c++) {
        suma = suma + v[c];
        if (suma > mejor) {
            mejor = suma;
        }
        if (suma < 0) {
            suma = 0;
        }
    }
    return mejor;
}
```

C++: maximum-sum.cpp

Fijar dos filas

```cpp
int mayor_rectangulo(const vector<vector<int>>& m) {
    int n = m.size();
    int mejor = m[0][0];
    for (int arriba = 0; arriba < n; arriba++) {
        vector<int> col(n, 0);
        for (int abajo = arriba; abajo < n; abajo++) {
            for (int c = 0; c < n; c++) {
                col[c] = col[c] + m[abajo][c];
            }
            int s = mejor_tramo(col);
            if (s > mejor) {
                mejor = s;
            }
        }
    }
    return mejor;
}
```

C++: maximum-sum.cpp

La lectura

```cpp
int main() {
    int n;
    scanf("%d", &n);
    vector<vector<int>> m(n, vector<int>(n));
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &m[i][j]);
        }
    }
    printf("%d\n", mayor_rectangulo(m));
    return 0;
}
```

¿Cabe en int?

La mayor suma posible es $127 \cdot 100^2 = 1\,270\,000$.

Trampas

Todo negativo

Con $N = 1$ y la celda -5, la respuesta es -5: el rectángulo tiene al menos una celda. Si `mejor` arranca en 0, sale 0.

Leer por líneas

La muestra misma reparte la matriz en líneas de 8, 5 y 3 números. Un programa que lee N números por línea falla con la muestra.

`col` sin reiniciar

col vuelve a ceros cada vez que cambia `arriba`; si no, arrastra filas de arriba que ya no están en el rectángulo.

Variante para pensar

Una matriz que no es cuadrada

La matriz tiene $f = 10$ filas y $c = 10\,000$ columnas. ¿Qué conviene fijar: dos filas o dos columnas?

Pista

Fijar dos filas cuesta $O(f^2 c)$ y fijar dos columnas, $O(c^2 f)$. Ponga los números.

¿Cuánto cuesta la solución?

De qué depende

Del lado N de la matriz, con $N \leq 100$: la entrada trae N^2 números.

Cuántas veces corre cada ciclo

`arriba` toma N valores y `abajo` a lo más N por cada uno: menos de N^2 pares de filas. Por cada par, N vueltas ponen al día *col* y otras N hace el recorrido de Kadane.

Total, contra el límite

$O(N^3)$, del orden de 10^6 pasos con $N = 100$, y $\Theta(N^2)$ de memoria. La tabla P daba $O(N^4)$, unas $2{,}6 \cdot 10^7$ consultas, y sumar celda por celda $O(N^6)$.

El juez da 3 segundos. Con una matriz aleatoria de 100×100 la solución tarda 0,06 s en Python 3.14 y menos de 0,01 s en C++ (Ryzen 5 3600).

Contenido

1 El mejor tramo de un arreglo

2 Rectángulos en una matriz

3 Recursión sobre un árbol
- Solución: UVa 839

Primera idea: mirar cada barra sola

Pregunta

La línea 1 es `0 2 0 4`. ¿Se puede decidir si está en equilibrio con esos cuatro números?

Respuesta

No: $0 \cdot 2 = 0 \cdot 4$ siempre da verdadero, y los pesos de verdad están en las líneas de abajo. La barra necesita, de cada submóvil, dos cosas: cuánto pesa y si está en equilibrio por dentro.

La función

`movil` lee una barra. Por cada extremo con peso 0 se llama a sí misma, que lee todas las líneas de ese submóvil y devuelve su peso y si está en equilibrio. Con los dos pesos de verdad compara $W_l D_l$ con $W_r D_r$ y devuelve $W_l + W_r$ y el resultado.

Caso base y caso recursivo

Caso base

Un extremo con objeto: su peso viene en la línea y no hay nada que leer.

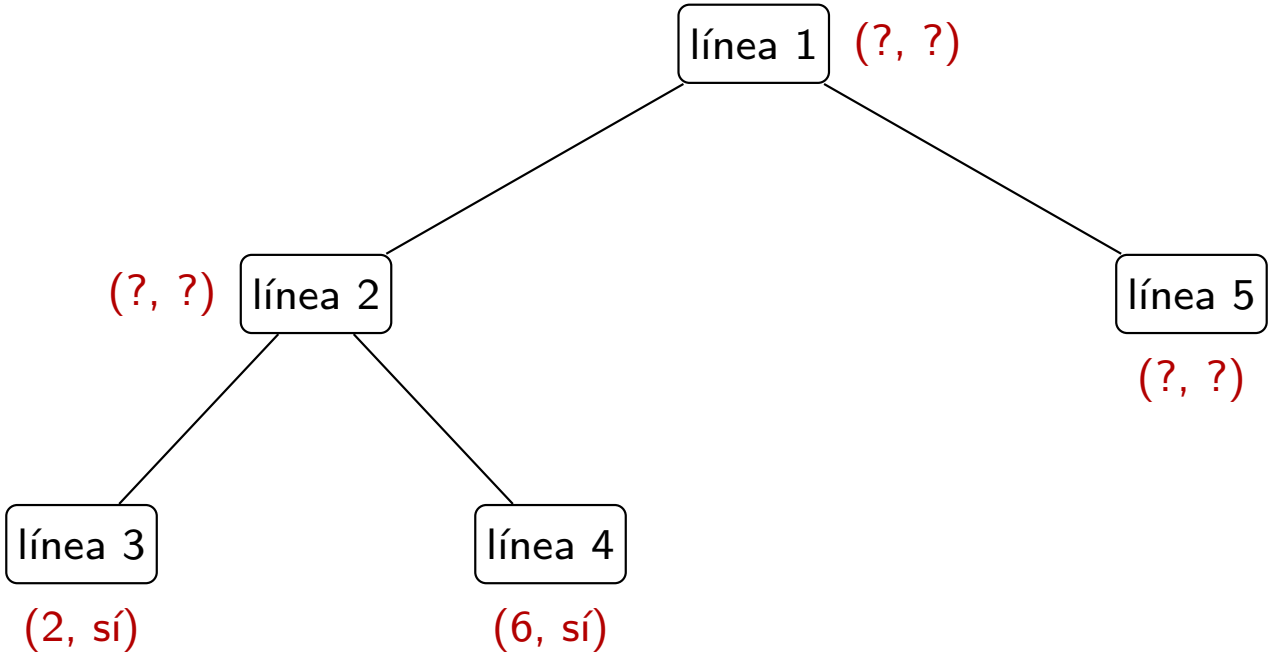
Caso recursivo

Un extremo con peso 0: una llamada que lee el submóvil completo. El submóvil tiene menos barras que el móvil, así que las llamadas terminan.

El árbol de llamadas es el móvil

Una llamada por barra, y cada llamada hace tantas llamadas como ceros tenga su línea: ninguna, una o dos.

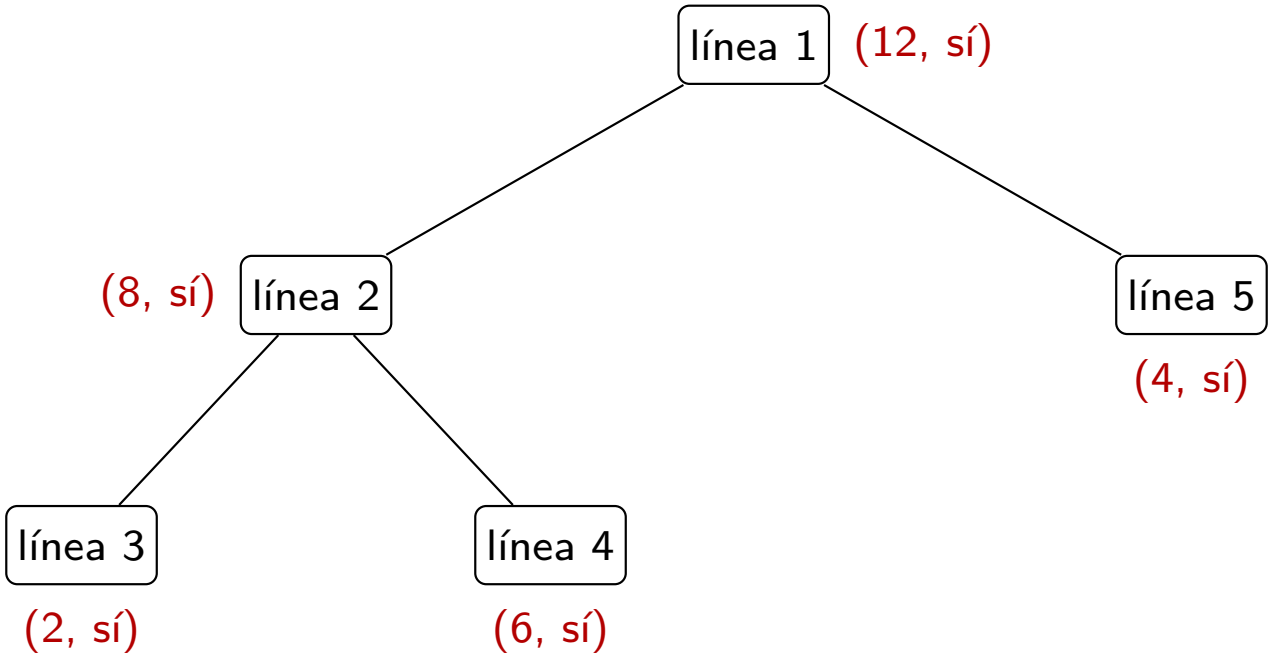
Lo que devuelve cada llamada



Pregunta

Las hojas ya devolvieron su par (peso, equilibrio). Complete los otros tres, de abajo hacia arriba.

Lo que devuelve cada llamada



Respuesta

Línea 2: $2 \cdot 3 = 6 \cdot 1$, pesa 8. Línea 5: $1 \cdot 6 = 3 \cdot 2$, pesa 4. Línea 1: con los pesos de verdad, $8 \cdot 2 = 4 \cdot 4$; todo el móvil pesa 12 y la salida es `YES`.

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	`0 2 0 4`	
2	línea 1, línea 2	`0 3 0 1`	
3	línea 1, línea 2, línea 3	`1 1 1 1`	(2, sí)
4	?	?	?
5	línea 1, línea 2		?
6	?	?	?
7	línea 1		?

Pregunta

¿Cuál es el mayor número de marcos de `movil` en la pila? ¿Qué tiene que ver con el dibujo del móvil?

La pila mientras se lee la muestra

Paso	Pila, del fondo al tope	Lee	Devuelve
1	línea 1	0 2 0 4	
2	línea 1, línea 2	0 3 0 1	
3	línea 1, línea 2, línea 3	1 1 1 1	(2, sí)
4	línea 1, línea 2, línea 4	2 4 4 2	(6, sí)
5	línea 1, línea 2		(8, sí)
6	línea 1, línea 5	1 6 3 2	(4, sí)
7	línea 1		(12, sí)

Respuesta

Tres marcos, en los pasos 3 y 4: tantos como niveles tiene el móvil. Una línea se lee cuando su llamada entra a la pila, y una llamada sale solo cuando ya leyó todo lo que cuelga de ella.

Python: not-so-mobile.py

La función: devuelve el peso, el equilibrio y dónde sigue la entrada

```
def movil(datos, pos):
    # lee la barra que empieza en datos[pos] y todo lo que cuelga de ella;
    # devuelve el peso, si está en equilibrio y dónde sigue la entrada
    wl = int(datos[pos])
    dl = int(datos[pos + 1])
    wr = int(datos[pos + 2])
    dr = int(datos[pos + 3])
    pos = pos + 4
    equilibrio = True
    if wl == 0:
        wl, eq_izq, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_izq
    if wr == 0:
        wr, eq_der, pos = movil(datos, pos)
        equilibrio = equilibrio and eq_der
    if wl * dl != wr * dr:
        equilibrio = False
    return wl + wr, equilibrio, pos
```

Python: not-so-mobile.py

Los casos

```
def main():
    sys.setrecursionlimit(10000)
    datos = sys.stdin.read().split()
    casos = int(datos[0])
    pos = 1
    for c in range(casos):
        peso, equilibrio, pos = movil(datos, pos)
        if c > 0:
            print("")
        if equilibrio:
            print("YES")
        else:
            print("NO")
```

sys.setrecursionlimit

Un móvil con más de mil niveles pasaría el límite de Python. El enunciado no da la altura máxima, y subir el límite a 10 000 cubre ese caso.

C++: not-so-mobile.cpp

La función: devuelve el peso y apaga equilibrio si una barra falla

```cpp
long long movil(bool& equilibrio) {
    long long wl, dl, wr, dr;
    scanf("%lld %lld %lld %lld", &wl, &dl, &wr, &dr);
    if (wl == 0) {
        wl = movil(equilibrio);
    }
    if (wr == 0) {
        wr = movil(equilibrio);
    }
    if (wl * dl != wr * dr) {
        equilibrio = false;
    }
    return wl + wr;
}
```

C++: not-so-mobile.cpp

Los casos

```
int main() {
    int casos;
    scanf("%d", &casos);
    for (int c = 0; c < casos; c++) {
        bool equilibrio = true;
        movil(equilibrio);
        if (c > 0) {
            printf("\n");
        }
        if (equilibrio) {
            printf("YES\n");
        } else {
            printf("NO\n");
        }
    }
    return 0;
}
```

Trampas

Dejar de leer cuando ya se sabe que es `NO`

Si la función no lee el submóvil derecho cuando el izquierdo ya falló, sus líneas quedan sin consumir y el caso siguiente arranca en mitad de este. Con los seis casos de `2.in`:

	1	2	3	4	5	6
esperado	NO	YES	NO	NO	NO	YES
sin leer el resto	NO	YES	YES	YES	NO	YES

Trampas

Las líneas en blanco

Se lee por palabras: los saltos de línea y las líneas vacías no cuentan.

La salida

La línea en blanco va entre dos casos, no después del último.

Pesos sin cota

El enunciado no acota los pesos ni las distancias, y la comparación multiplica un peso acumulado por una distancia: en C++ van en `long long`.

Revisar solo la barra de arriba

Un móvil cuya barra de arriba está en equilibrio y alguna de abajo no, se responde con `NO`: el caso 5 de `2.in` es así.

Variante para pensar

Cuántas barras fallan

En lugar de `YES` o `NO`, imprima cuántas barras del móvil no están en equilibrio. En el caso 5 de `2.in` son 2.

Pista

La función devuelve el peso y un contador en lugar del par. El contador de una barra es el de cada submóvil más 1 si ella misma falla.

¿Cuánto cuesta la solución?

De qué depende

Del número n de barras del móvil, que el enunciado no acota.

Cuántas llamadas hay

Una por barra, n por caso, sea cual sea la forma del móvil, y cada una hace un trabajo fijo: leer cuatro números, comparar dos productos y sumar dos pesos.

Total, contra el límite

$\Theta(n)$ por caso, igual que leer la entrada. La pila llega a la altura del móvil: unos $\log_2 n$ marcos si el árbol está lleno, n si es una cadena.

Lo que se puede acabar es la pila: una cadena de más de mil barras pasa el límite de Python, que la solución sube con `sys.setrecursionlimit`.